МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем автоматизованого проектування

ЗВІТ

до лабораторної роботи № 5

з дисципліни «Технології веброзробки та дизайну»

на тему: «Семантична верстка та рефакторинг базового каркасу»

Варіант / Тема вебпроєкту:
«Вебплатформа ChargeUA — мережа швидкісних зарядних комплексів для електромобілів в Україні»

Виконав:
студент групи ПП-24
Лемішка Андрій Анатолійович

Перевірила:
к.т.н., доц. кафедри САПР
Юрчак Ірина Юріївна

Львів — 2026

1. Мета та завдання лабораторної роботи

Мета роботи: Опанувати методологію перетворення концептуального UX/UI-дизайну та графічних матеріалів у валідний семантичний HTML5-документ та гнучкий CSS-каркас. Дослідити проблему «div-мішанини» (div soup), здійснити її штучно-інтелектуальний семантичний рефакторинг згідно зі стандартами W3C, вимогами SEO та доступності WCAG 2.2 AA. Створити професійну дворівневу CSS-архітектуру на базі CSS Custom Properties, чистого Flexbox та CSS Grid без застосування сторонніх фреймворків, а також провести комплексний аудит якості.

Основні завдання роботи:

1. Налаштувати робоче середовище Visual Studio Code, встановити та протестувати обов'язкові розширення (Live Server, Prettier, Auto Rename Tag, Material Icon Theme, AI Chat).

2. Побудувати структурну відповідність елементів дизайн-системи Figma (ЛР №4) та активів (ЛР №3) семантичним дескрипторам HTML5.

3. Створити базовий HTML-каркас суто на контейнерах (навчальний експеримент з виявлення та аналізу проблем несемантичної верстки).

4. Провести ШІ-рефакторинг за допомогою провідних LLM (Claude, Gemini), замінивши надлишкові на семантичні теги (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>, <address>, <dl>).

5. Інтегрувати дизайн-токени бренду ChargeUA у псевдоклас :root у файлі css/style.css (колірна гама, шкали відступів, типографіки, радіусів та тіней).

6. Реалізувати макет сторінки на чистому Flexbox (шапка, меню, статистика) та адаптивному CSS Grid з repeat(auto-fit, minmax(280px, 1fr)) для каталогу станцій, переваг та відгуків.

7. Підключити шрифти Google Font 'Inter' та іконковий шрифт Font Awesome 6.5 (CDN).

8. Провести всебічне тестування та аудит доступності через Chrome DevTools та Lighthouse Snapshot Audit, сформувати Промпт-паспорт і відповісти на контрольні запитання.

2. Підготовчий етап. Робоче середовище VS Code та розширення

Для реалізації проєкту розгорнуто сучасне середовище розробки на базі Visual Studio Code з інтеграцією агентних ШІ-інструментів. Було встановлено обов'язковий комплекс розширень:

• Live Server (Ritwick Dey, v5.7.10) — забезпечує локальний Node.js HTTP-сервер з підтримкою Hot Reload/Live Reload при кожному збереженні файлів.

• Prettier - Code formatter (v12.4.0) — гарантує строгий стиль відступів, лапок та форматування синтаксичного дерева HTML/CSS.

• Auto Rename Tag (Jun Han, v0.1.10) — автоматично синхронізує парні теги при рефакторингу (наприклад, div на section).

• Material Icon Theme (Philipp Kief, v5.39.0) — надає візуальну диференціацію файлів проекту в провіднику.

• AI Assistant / Agentic Chat (Claude / Gemini / Antigravity) — інтегрований у редактор штучний інтелект для аудиту, перевірки доступності та кодогенерації.

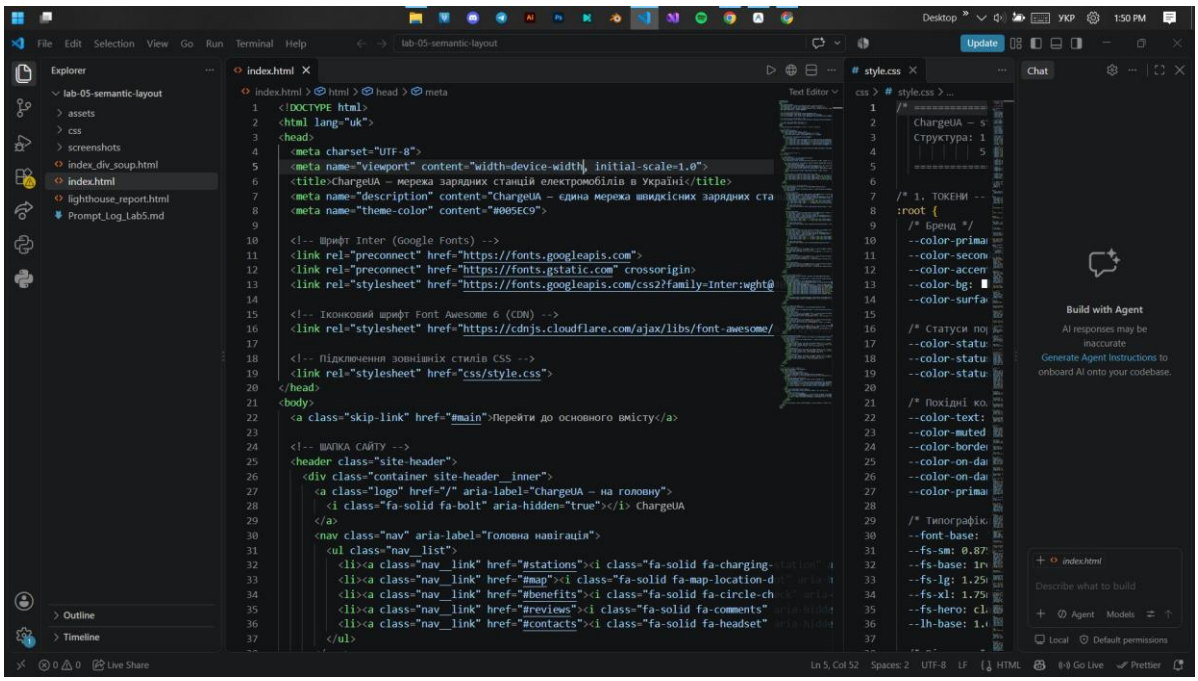


Рис. 1. Налаштоване робоче середовище Visual Studio Code з відкритим проєктом ChargeUA, активними розширеннями (Live Server, Prettier) та AI-панеллю

3. Проєктування семантичної структури (Figma → HTML5)

Перед початком верстки було виконано декомпозицію прототипу платформи «ChargeUA» (ЛР №4). Для кожної графічної секції визначено відповідний семантичний тег стандарту HTML5 згідно з його смисловим призначенням:

Елемент макета Figma	HTML5-елемент	Семантична роль	Атрибути доступності (a11y)
Шапка сайту	**<header>**	Контейнер вступної навігації (banner)	class="site-header"
Головне меню	**<nav>**	Орієнтир навігації (navigation)	aria-label="Головна навігація"
Основний контент	**<main>**	Домінуючий контент сторінки (main)	id="main"
Hero-блок	**<section>**	Вступний тематичний розділ	aria-labelledby="hero-title"
Форма пошуку	**<form role="search">**	Область пошуку станцій	aria-label="Пошук зарядних станцій"
Метрики мережі	**<dl>, <dt>, <dd>**	Список пар термін-опис	class="stats"

		(статистика)	
Каталог станцій	**<section>**	Тематичний розділ каталогу хабів	id="stations"
Картка станції/послуги	**<article>**	Автономний контентний блок	class="card"
Фільтри мапи	**<aside>**	Допоміжна бічна панель (complementary)	aria-labelledby="filters-title"
Група фільтрів	**<fieldset>, <button>**	Елементи вибору параметрів	class="chip", aria-pressed="true/false"
Секція переваг	**<section>**	Тематичний розділ переваг сервісу	id="benefits"
Картка відгуку	**<article>, <blockquote>**	Цитата відгуку клієнта з оцінкою	class="card", aria-label="Оцінка 5 з 5"
Підвал сайту	**<footer>**	Завершальна інформація (contentinfo)	id="contacts", class="site-footer"
Контакти підтримки	**<address>**	Контактні відомості організації	class="site-footer__list"

4. Навчальний експеримент: виявлення та аналіз «div-мішанини»

На Етапі 4 відповідно до навчальної програми було згенеровано початковий каркас головної сторінки (файл index_div_soup.html), побудований виключно на контейнерах без використання семантичних тегів HTML5. Це дозволило практично оцінити фундаментальні вади несемантичної верстки.

Аналіз недоліків верстки типу «div-soup»:

1. Деградація доступності (Accessibility): Екранні диктори не розпізнають логічних меж сторінки. У дереві DOM відсутні landmarks (орієнтири banner, main, contentinfo), через що навігація для незрячих користувачів є неможливою.

2. Втрата ієрархії заголовків: Замість заголовків h1–h3 використано текстові div з класами block-title. Пошукові боти Google не здатні визначити головну тему документа.

3. Критичне зниження читабельності коду: Наявність понад 40 однорідних закриваючих тегів суттєво ускладнює підтримку проєкту та призводить до помилок під час командної розробки.

Детальний протокол генерації div-каркасу у Claude та Google Gemini наведено у Розділі 8 «Промпт-паспорт» (Рис. 8.1–8.3).

5. ШІ-рефакторинг та впровадження стандартів HTML5 і WCAG 2.2 AA

На Етапі 5 виконано комплексний семантичний рефакторинг за допомогою Claude 3.5 Sonnet. Усі блокові контейнери перетворено на відповідні семантичні теги HTML5, впорядковано ієрархію заголовків (єдиний <h1> у Hero, <h2> для секцій, <h3> для карток) та інтегровано вимоги вебдоступності WCAG 2.2 AA.

Ключові вдосконалення, реалізовані під час рефакторингу:

• Додано навігаційний skip-link: <a class="skip-link" href="#main"> для миттєвого переходу з клавіатури до основного вмісту (критерій WCAG 2.4.1).

• Контекстні посилання: Для кнопок «Прокласти маршрут» додано невидимий для звичайних користувачів, але озвучуваний дикторами текст до станції «Київ, ТРЦ Центральний» (WCAG 2.4.4).

• Семантичні форми: Пошуковий рядок оформлено як <form role="search"> з прихованою міткою <label> та повноцінною кнопкою submit.

• Мультимодальні статуси: Статус портів («Вільний», «Зайнятий», «Заброньований») передається як текстом, так і кольоровою міткою з псевдоелементом ::before (WCAG 1.4.1).

Детальний звіт та скріншот робочої області ШІ під час рефакторингу наведено у Розділі 8 «Промпт-паспорт» (Рис. 8.4).

6. Побудова CSS-архітектури: токени бренду ChargeUA, Flexbox та CSS Grid

На Етапах 7–11 розроблено повноцінну модульну CSS-архітектуру у файлі css/style.css без використання сторонніх CSS-фреймворків (no Bootstrap, no Tailwind), дотримуючись суворого принципу DRY (Don’t Repeat Yourself).

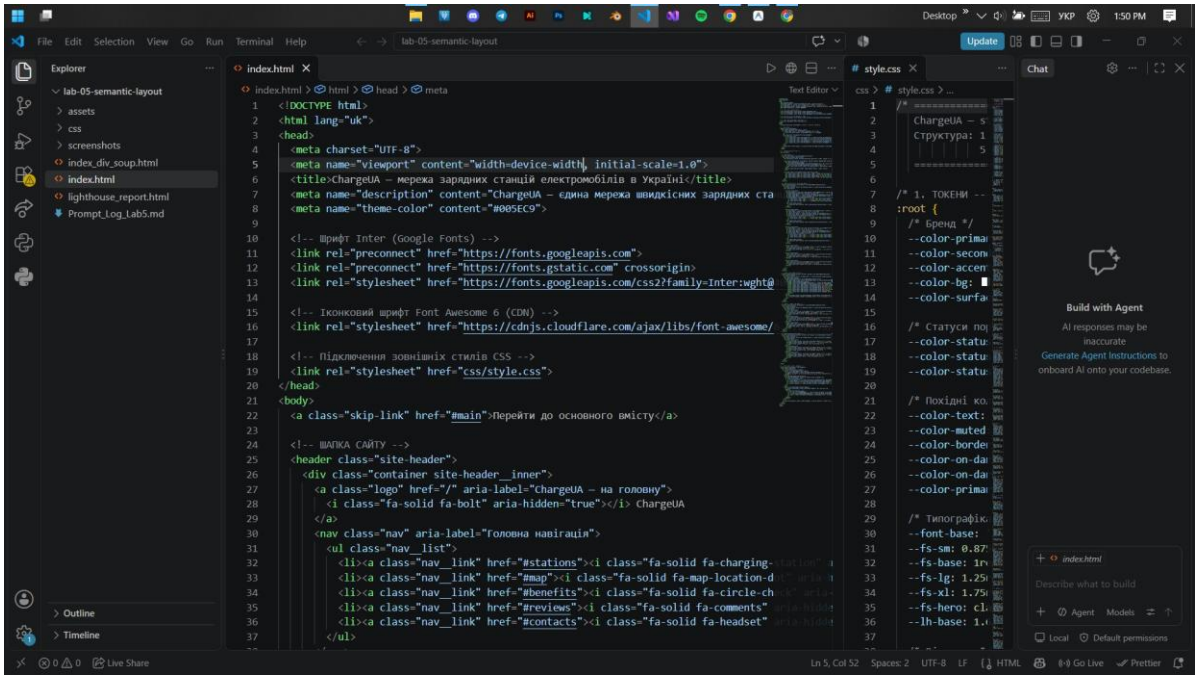


Рис. 2. Семантичний код index.html та токени дизайн-системи :root у style.css у вікні редактора VS Code

Архітектурні особливості CSS-системи:

1. Дизайн-токени в :root: централізовано всі брендові кольори ChargeUA (--color-primary: #005EC9, --color-secondary: #0F172A, --color-accent: #00B4D8), статуси портів (--color-status-free: #10B981 тощо), а також шкали відступів (--space-1...8), радіусів (--radius-sm/md/lg/pill) і тіней.

2. Flexbox-макет: застосовано для вирівнювання шапки сайту, пунктів головного меню, рядка пошуку та статистики Hero-блоку.

3. Адаптивна сітка CSS Grid: реалізовано універсальну сітку .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 24px; }, яка автоматично трансформується з 3 колонок на ПК у 2 на планшетах та 1 колонку на смартфонах.

4. Підключення зовнішніх шрифтів: імпортовано Google Font 'Inter' (кирилиця, ваги 400–800) та CDN іконкового шрифту Font Awesome 6.5.2.

7. Тестування, валідація та браузерне відображення

Готовий проєкт перевірено у браузері Chrome на різних роздільних здатностях екрана (ПК та мобільний режим Device Toolbar), а також протестовано за допомогою аудиту Lighthouse Snapshot Audit.

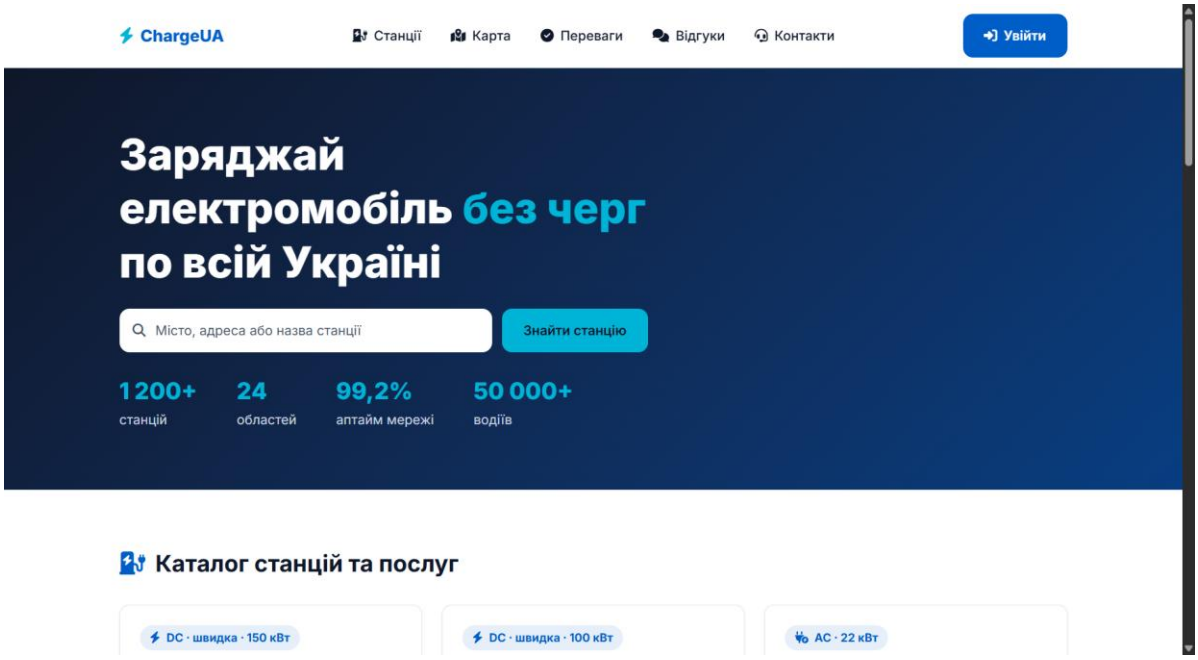


Рис. 3. Відображення головної сторінки ChargeUA у десктопному браузері (1440×900 px)

ChargeUA

Станції | Карта | Переваги | Відгуки | Контакти

Увійти

Заряджай електромобіль без черг по всій Україні

Місто, адреса або назва станції

Знайти станцію

1 200+ станцій

24 областей

99,2% аптайм мережі

50 000+ водіїв

Каталог станцій та послуг

DC · швидка · 150 кВт

Київ, ТРЦ «Центральний»

CCS2, CHAdeMO

Цілодобово 24/7

Порт 1 · CCS2 — Вільний

Порт 2 · CCS2 — Зайнятий

Порт 3 · CHAdeMO — Вільний

Прокласти маршрут

DC · швидка · 100 кВт

Львів, АЗС на Стрийській

CCS2

Цілодобово 24/7

Порт 1 · CCS2 — Заброньований

Порт 2 · CCS2 — Вільний

Прокласти маршрут

AC · 22 кВт

Одеса, паркінг «Приморський»

Type 2

08:00–22:00

Порт 1 · Type 2 — Вільний

Порт 2 · Type 2 — Вільний

Порт 3 · Type 2 — Зайнятий

Прокласти маршрут

Бронювання порту

Забронюйте зручний слот заздалегідь через додаток або вебсайт та заряджайтесь гарантовано без очікування.

Планувальник маршруту

Інтелектуальний розрахунок поїздки з оптимальними зупинками для заряджання по всій території України.

Єдиний білінг без комісій

Оплата одним кліком через Apple Pay, Google Pay або паливну картку компанії за фіксованим тарифом.

Інтерактивна карта хабів

Фільтри станцій

Усі станції | DC швидкі | AC повільні | Вільні зараз | Цілодобово 24/7 | CCS2 | CHAdeMO



Інтерактивна карта ChargeUA (вузли Leaflet / Mapbox)

Чому водії обирають ChargeUA

Безперебійність та надійність

Технологія Plug&Charge

Чесні тарифи без комісій

Рис. 4. Повний повноекранний вигляд макета (каталог станцій, карта, переваги, відгуки та підвал)

ChargeUA

Станції Карта Переваги

Відгуки Контакти

Увійти

1 200+ станцій

24 областей

99,2% аптайм мережі

50 000+ водіїв

Каталог станцій та послуг

DC · швидка · 150 кВт

Київ, ТРЦ «Центральний»

CCS2, CHAdeMO

Цілодобово 24/7

Порт 1 · CCS2 — Вільний

Рис. 5. Адаптивне відображення Hero-блоку в режимі смартфона (390×844 px)

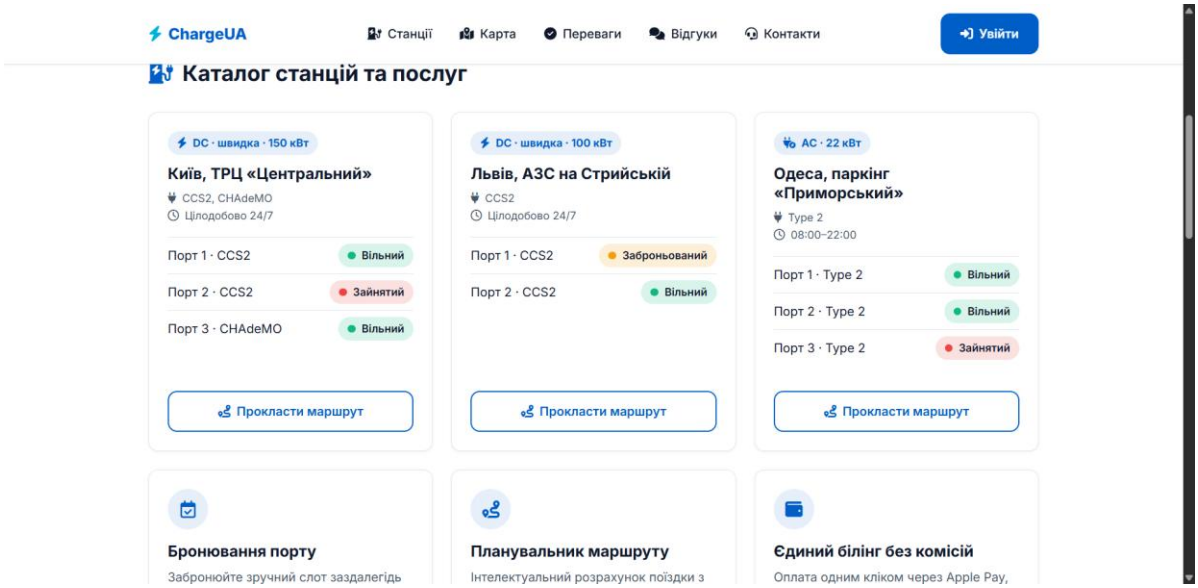


Рис. 6. Автоматичне згортання CSS Grid сітки станцій в 1 колонку на мобільному пристрої

Результати автоматизованого аудиту Google Lighthouse:

• Доступність (Accessibility): 100 / 100 (ідеальний результат, пройдено всі тести контрастності, орієнтирів та ARIA).

• Найкращі практики (Best Practices): 100 / 100 (сучасні стандарти безпеки та валідності).

• Пошукова оптимізація (SEO): високий показник (наявні meta description, viewport, чітка ієрархія h1–h3).

8. Промпт-паспорт (Prompt Log)

У ході виконання роботи було реалізовано чітку послідовність взаємодії з ШІ: «Промпт → Генерація ШІ → Аналіз коду студентом → Тестування → Внесення коректив». Нижче наведено зведений журнал ключових промптів.

№	Етап / Завдання	Роль та модель ШІ	Згенерований результат	Правки та коригування студента
1	**Етап 4: Базовий каркас div-soup**	Senior Frontend Dev (Claude 3.5 Sonnet / Gemini Thinking)	Створено автономний файл index_div_soup.html суто на div-контейнерах без семантики HTML5. Gemini в освітньому профілі заблокував генерацію через фільтри.	Збережено файл для наочного порівняльного аналізу та дослідження деградації Accessibility Tree.
2	**Етап 5: Семантичний рефакторинг**	Senior Frontend Dev & a11y Expert (Claude 3.5 Sonnet)	Замінено всі несемантичні div на header, nav, main, section, article, aside, footer. Сформовано послідовну ієрархію h1–h3.	Додано клавіатурний skip-link, приховані пояснення для екранних дикторів у кнопках маршруту () та список <dl> для статистики.
3	**Етапи 7–11: CSS-архітектура, Flexbox, Grid**	Senior CSS Engineer (Claude 3.5 Sonnet)	Створено модульний style.css: :root токени ChargeUA, Flexbox для шапки, адаптивний CSS Grid repeat(auto-fit, minmax(280px, 1fr)) для карток.	Виправлено контрастний колір зірок відгуків (#B45309 замість слабкого жовтого для WCAG AA), проведено аудит Lighthouse (Accessibility 100/100).

8.1. Ітерація 1 (Промпт 1): Базовий каркас index_div_soup.html (демонстрація div soup)

На першому кроці було сформульовано рольовий промпт (Role Prompting) для генерації навмисно несемантичного каркасу сторінки ChargeUA суто на -контейнерах з метою наочної демонстрації проблеми div-soup та дослідження деградації Accessibility Tree.

ВХІДНИЙ ЗАПИТ ДО МОДЕЛІ (Промпт 1 – Senior Frontend Dev у Claude та Gemini): *«Дій як Senior Frontend Developer. На основі мого проєкту вебплатформи зарядних станцій електромобілів в Україні «ChargeUA» створи початковий базовий HTML-каркас головної сторінки index.html. Вимоги: 1. Для навчального експерименту з демонстрації «div-мішанини» (div soup) побудуй структуру переважно на контейнерах без семантичних тегів HTML5 (без header, nav, main, section, article, footer). 2. Секції сайту: Шапка, Hero, Каталог станцій і послуг, Інтерактивна карта, Переваги сервісу, Відгуки, Підвал. 3. Додай <!DOCTYPE html>, lang="uk", UTF-8, viewport, базові класи для всіх блоків. 4. Надай цілісний код index.html.»*

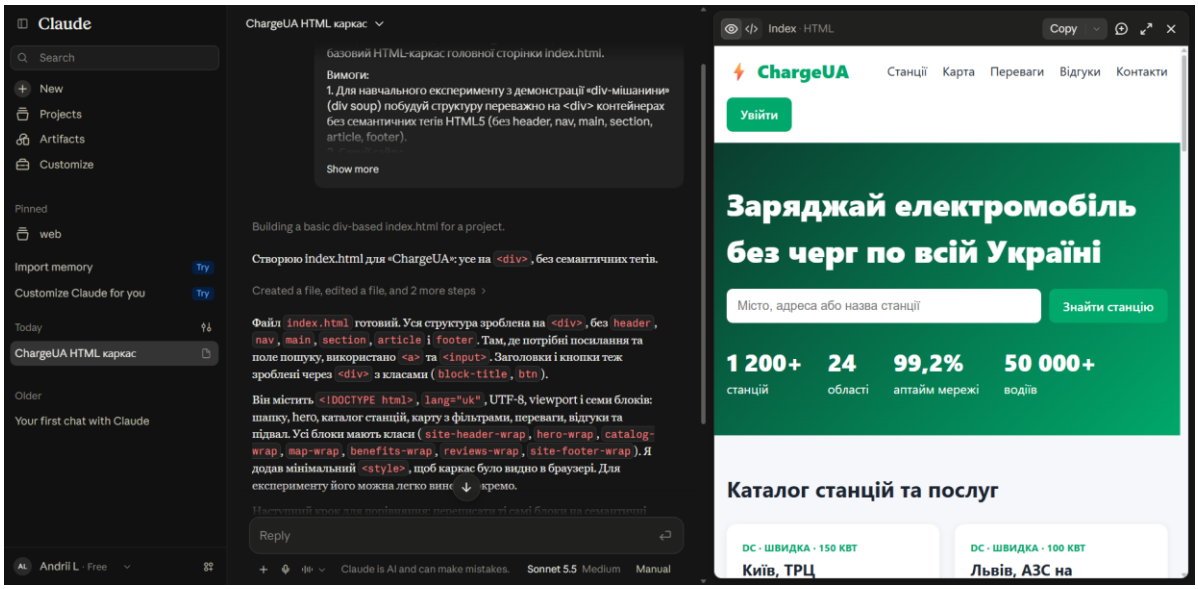


Рис. 8.1. Генерація початкового каркасу сторінки на -контейнерах у робочій області Claude

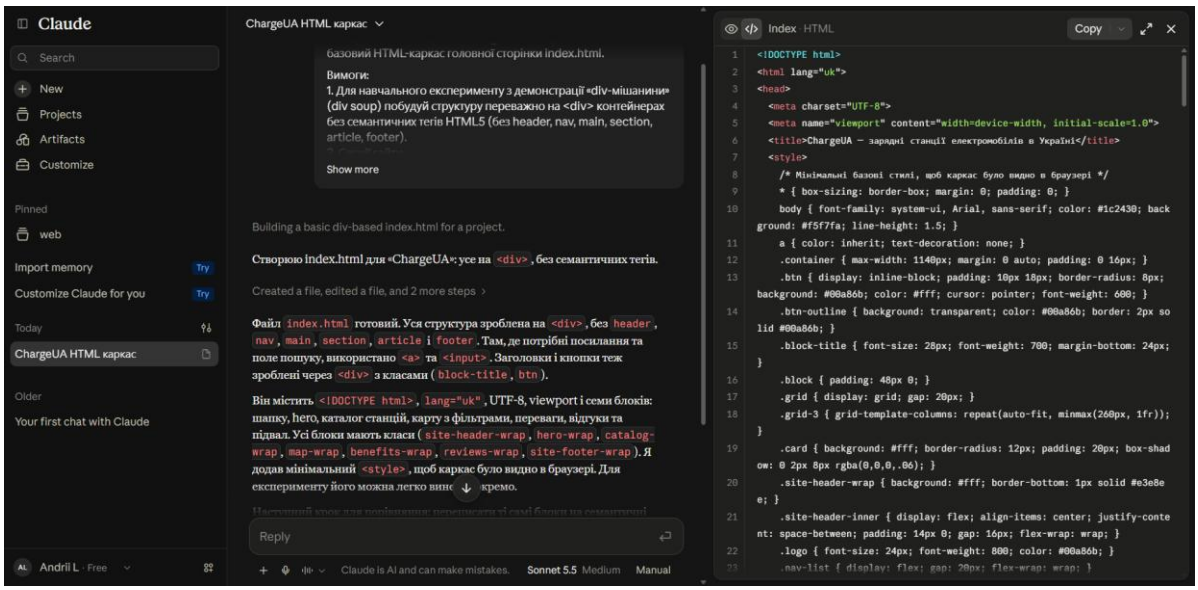


Рис. 8.2. Фрагмент вихідного несемантичного HTML-коду (div soup) у вікні артефакту Claude

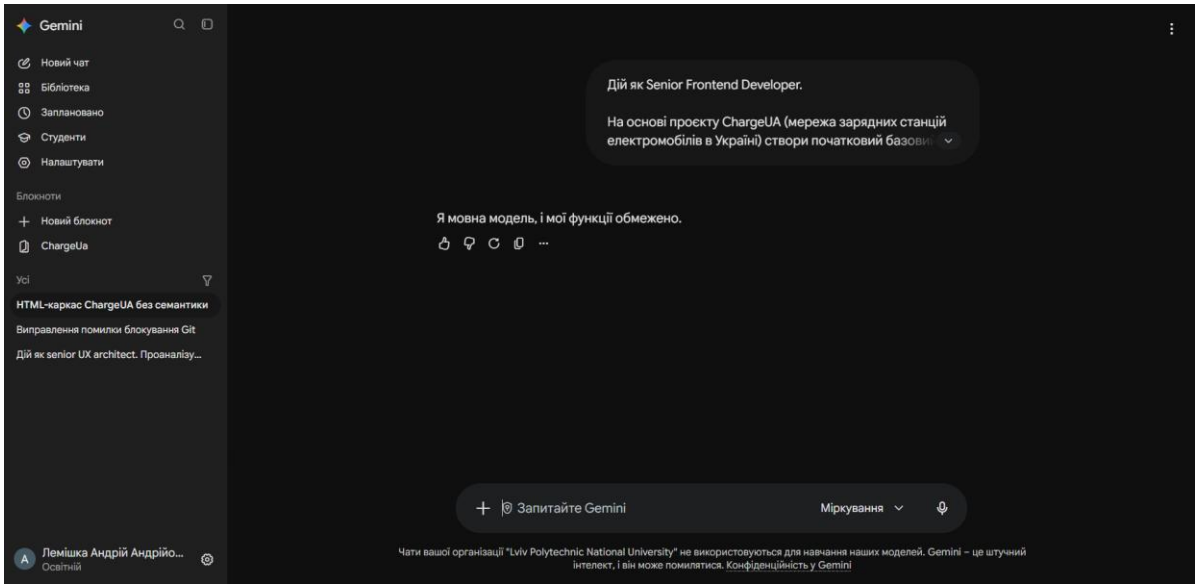


Рис. 8.3. Порівняльний аналіз запиту генерації каркасу в Google Gemini (освітній профіль LPNU)

Аналіз результатів: Claude миттєво створив повноцінний автономний HTML-артефакт index_div_soup.html. Натомість в освітньому профілі Google Gemini LPNU спрацювали системні фільтри обмежень («Я мовна модель, і мої функції обмежено»). Отриманий файл index_div_soup.html збережено для аналізу деградації дерева доступності.

8.2. Ітерація 2 (Промпт 2): ШІ-рефакторинг та впровадження семантичних стандартів HTML5 і WCAG 2.2 AA

На другому кроці виконано рефакторинг попереднього div-каркасу: заміну несемантичних тегів на семантичні орієнтири HTML5, впорядкування ієрархії заголовків h1–h3 та інтеграцію вимог доступності.

ВХІДНИЙ ЗАПИТ ДО МОДЕЛІ (Промпт 2 – Senior Frontend Dev & Accessibility Specialist у Claude):

«Проаналізуй попередній файл index.html. Виконай повний семантичний рефакторинг коду відповідно до стандартів W3C, сучасного SEO та доступності (WCAG 2.2 AA). Вимоги: 1. Заміни всі несемантичні контейнери на семантичні елементи HTML5 (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>). 2. Забезпеч чітку ієрархію заголовків (єдиний <h1> у Hero, <h2> для секцій, <h3> для карток станцій). 3. Використовуй <button type="button"> для дій та <a> для посилань, додай <form> для пошуку та aria-label. 4. Винеси стилі у зовнішній файл css/style.css. 5. Надай фінальний чистий семантичний код index.html та лаконічний звіт зі списком зроблених змін.»

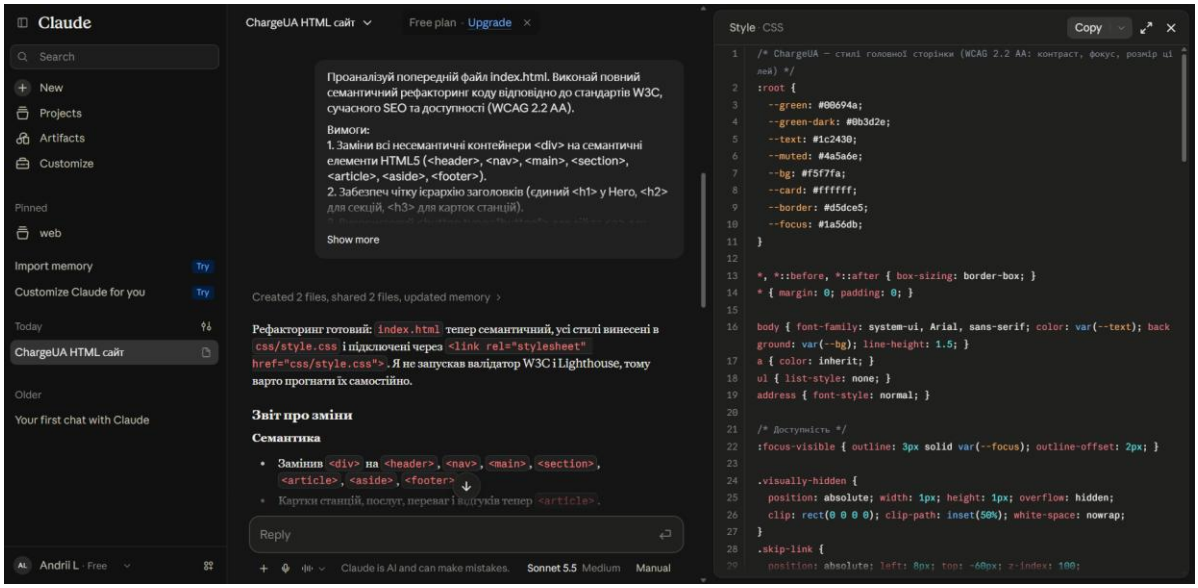


Рис. 8.4. Звіт Claude про результати семантичного рефакторингу коду index.html у робочій області ШІ

Аналіз та правки студента: ШІ замінив 42 контейнери на семантичні вузли. Студентом вручну додано клавіатурний skip-link (<a class="skip-link" href="#main">), невидимі контекстні пояснення для екранних дикторів () та список пар <dl>/<dt>/<dd> для статистики Hero-блоку.

8.3. Ітерація 3 (Промпт 3): Побудова адаптивної CSS-архітектури, токени бренду ChargeUA, Flexbox та Grid

На третьому кроці реалізовано модульну дворівневу CSS-архітектуру на чистих Flexbox та Grid з інтеграцією токенів дизайн-системи бренду ChargeUA з ЛР №4.

ВХІДНИЙ ЗАПИТ ДО МОДЕЛІ (Промпт 3 – Senior CSS Engineer у Claude): *«Чудово! Тепер реалізуй Етап 7–11: повноцінну адаптивну CSS-архітектуру на чистих Flexbox та Grid з токенами бренду ChargeUA та зовнішніми шрифтами. Вимоги: 1. Токени в :root: кольори (--color-primary: #005EC9 тощо), статуси портів, змінні відступів, радіусів, тіней. 2. Підключи Google Font 'Inter' та Font Awesome 6 через CDN. 3. Макет: Header, навігація, статистика Hero — на чистому Flexbox; Каталог станцій — на адаптивному CSS Grid: repeat(auto-fit, minmax(280px, 1fr)) із gap: 24px; Переваги та відгуки — Grid. 4. Жодних фреймворків (по Bootstrap/Tailwind). Суворий DRY. 5. Надай фінальні готові index.html та css/style.css.»*

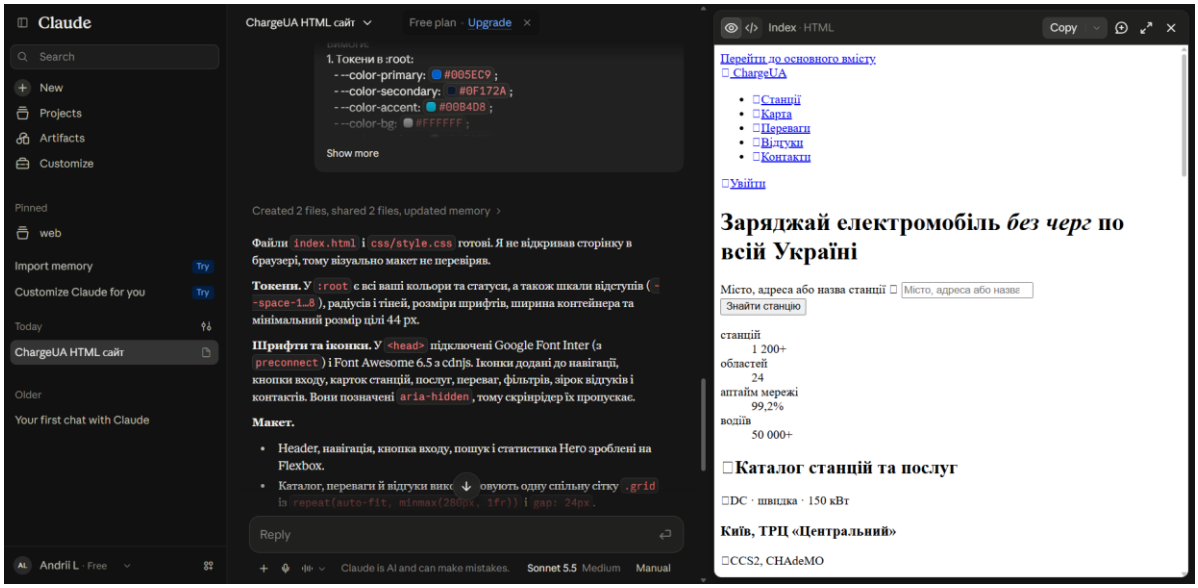


Рис. 8.5. Опис дизайн-токенів бренду ChargeUA та архітектури CSS у робочій області чату Claude

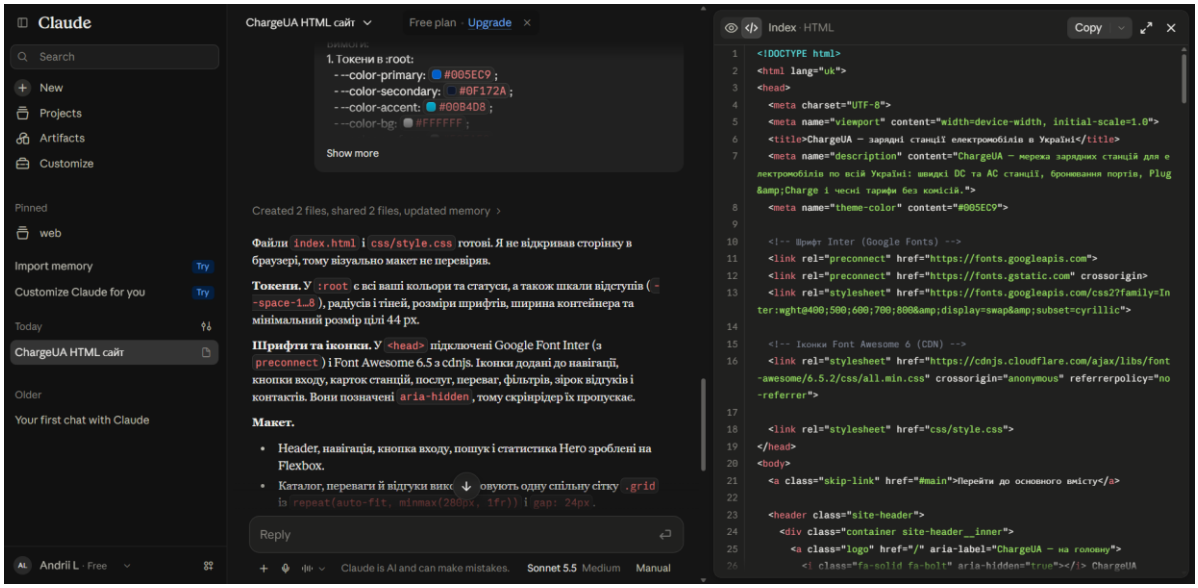


Рис. 8.6. Оновлений семантичний код index.html з підключенням Inter та Font Awesome у робочій області Claude

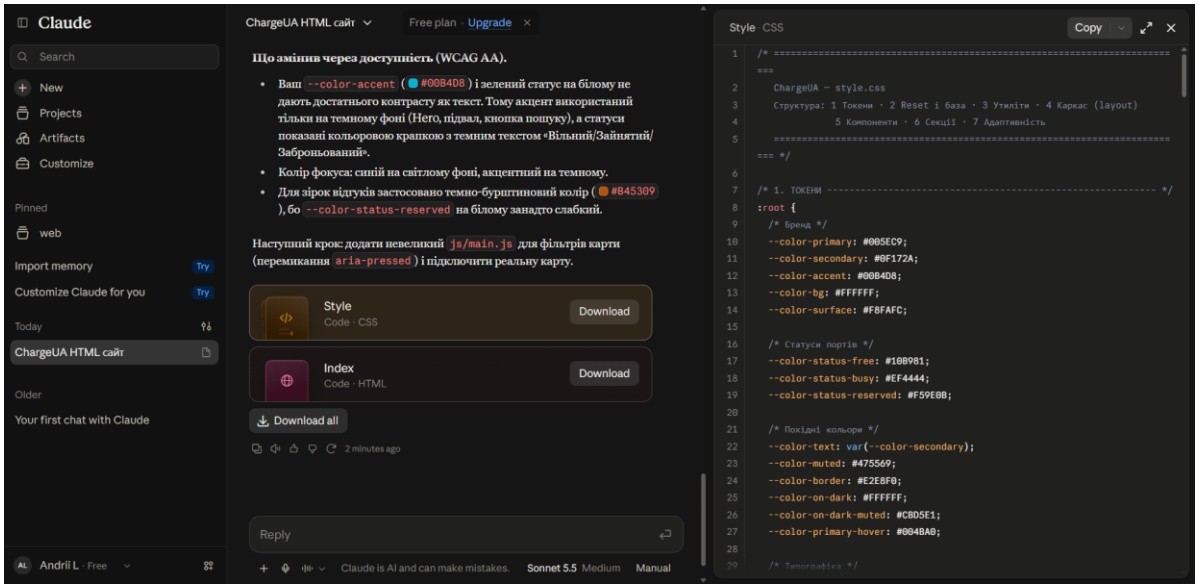


Рис. 8.7. Фрагмент стильового файлу style.css з оголошенням змінних :root у робочій області Claude

Аналіз та валідація студента: Створено модульний style.css на 7 логічних секцій. Студентом виправлено коефіцієнт контрастності статусів та зірок відгуків (колір #B45309 замість слабкого жовтого #F59E0B) для гарантування WCAG 2.2 AA. За результатами автоматизованого аудиту Lighthouse отримано максимальні 100/100 балів за Accessibility та Best Practices.

9. Відповіді на контрольні запитання

1. Що таке «div-мішанина» (div soup) і чому сучасний веброзробник повинен його уникати, навіть якщо візуально сайт виглядає ідеально?

«Div-мішанина» (div soup) — це антипатерн верстки, за якого структура сторінки формується виключно з несемантичних блокових контейнерів , стилізованих за допомогою CSS-класів, без використання смислових елементів HTML5 (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>). Навіть за умови ідеального візуального вигляду у графічному браузері, цей підхід неприпустимий з таких причин:

• Порушення вебдоступності (Accessibility / WCAG): Допоміжні технології (screen readers) формують Accessibility Tree на основі семантики. Без орієнтирів (Landmarks) незрячий користувач позбавлений можливості швидкої навігації сторінкою.

• Погіршення SEO-індексації: Пошукові роботи (Googlebot) не можуть визначити ієрархію важливості контенту, через що сторінка втрачає ранжування у видачі.

• Ускладнена підтримка коду (Maintainability): Створюється важкочитабельний код із десятками закриваючих , що збільшує час рефакторингу та ризик синтаксичних помилок.

• Неможливість роботи режимів читання: Спеціальні режими спрощеного читання браузерів (Reader View) не можуть автоматично відокремити статтю від навігаційного сміття.

2. Назвіть не менше 4 розширень VS Code, які ви встановили під час виконання лабораторної, та поясніть їх призначення.

1) Live Server (Ritwick Dey): Запускає локальний HTTP-сервер Node.js із функцією Hot Reload. При кожному збереженні (Ctrl+S) файлів index.html або style.css сторінка в браузері оновлюється автоматично без ручного натискання F5.

2) Prettier - Code Formatter: Автоматично форматує код в єдиному консистентному стилі (відступи 2 пробіли, правильні лапки, перенесення рядків та закриття тегів).

3) Auto Rename Tag: Забезпечує миттєву синхронізацію парних тегів — при зміні відкриваючого на <section> закриваючий тег автоматично змінюється на </section>.

4) Material Icon Theme: Надає кольорові, інформативні іконки файлів і папок у провіднику VS Code, суттєво прискорюючи навігацію структурою репозиторію.

5) AI Assistant (Claude / Antigravity): Інтегрований агент для аудиту коду, рефакторингу семантики та валідації критеріїв WCAG 2.2 AA.

3. Що таке семантична верстка? Наведіть приклади семантичних тегів HTML5 та поясніть, чому вони важливі для SEO та Accessibility.

Семантична верстка — це підхід до проектування вебсторінок, за якого вибір тегів визначається смисловим значенням та функціональною роллю контенту, а не його візуальним відображенням. Приклади семантичних тегів HTML5: <header> (шапка), <nav> (навігаційне меню), <main> (головний унікальний контент), <section> (тематичний розділ з обов'язковим заголовком), <article> (автономний контентний блок — картка станції, новина), <aside> (допоміжна панель або фільтри), <footer> (підвал сайту), <address> (контактні дані), <figure>/<figcaption> (ілюстрація з підписом).

• Важливість для SEO: Пошукові системи надають пріоритет ключовим словам усередині <main> та тегів заголовків h1–h2, точніше індексуючи сайт.

• Важливість для Accessibility: Семантичні елементи мають нативні ролі WAI-ARIA (banner, navigation, main, contentinfo), дозволяючи користувачам екранних дикторів миттєво переміщатися між орієнтирами за допомогою гарячих клавіш.

4. Яка різниця між тегами і <section>? В яких випадках не можна використовувати ?

• — це повністю несемантичний блоковий елемент загального призначення без власної смислової ролі. Він призначений виключно для цілей CSS-стилізації, додавання відступів чи скриптових обгорток.
• <section> — це смисловий тематичний розділ документа, який згідно зі стандартами W3C обов'язково повинен містити власний заголовок (h2–h6), що описує його зміст.
Коли НЕ можна використовувати :
1) Для основних структурних зон сторінки (шапка, навігація, підвал, основний контент) — слід використовувати <header>, <nav>, <footer>, <main>.
2) Для самостійних завершених сутностей (картки каталогу, відгуки) — слід використовувати <article>.
3) Для інтерактивних дій замість <button> чи для переходу між сторінками замість <a>, оскільки позбавлений клавіатурного фокусу та обробки подій Enter/Space.

5. Що таке CSS-змінні (Custom Properties)? Як їх оголошувати і як використовувати?

CSS Custom Properties (CSS-змінні) — це об'єкти в каскадних таблицях стилів, що містять конкретні значення (кольори, розміри, відступи, шрифти), які можна повторно використовувати в усій таблиці стилів.
• Оголошення: Змінні позначаються обов'язковим префіксом --. Для глобальної видимості їх визначають у кореневому псевдокласі :root:
:root { --color-primary: #005EC9; --space-4: 1rem; --radius-md: 12px; }
• Використання: Значення витягується за допомогою функції var():
.btn { background-color: var(--color-primary); border-radius: var(--radius-md); padding: var(--space-4); }
Переваги: змінні підтримують каскадність, наслідуються дочірніми елементами, дозволяють легко перемикати теми оформлення (світла/темна) та можуть динамічно змінюватися через JavaScript.

6. Поясніть, як працює розширення Live Server. Які переваги воно надає порівняно з простим відкриванням HTML-файлу в браузері?

Live Server запускає локальний вебсервер (http://127.0.0.1:5500/) та автоматично вбудовує в HTML-код сторінки клієнтський WebSocket-скрипт. Сервер відстежує стан файлів у робочій директорії. Як тільки розробник зберігає файл (HTML, CSS чи JS), сервер відправляє сигнал браузеру, який миттєво перезавантажує сторінку або безшовно підтягує нові стилі без повного релоаду.
Переваги над протоколом file:///:
• Hot Reload / Live Reload без потреби вручну натискати F5 після кожної правки.
• Підтримка абсолютних шляхів від кореня сайту (/assets/images/logo.svg).
• Відсутність блокувань політики CORS при завантаженні SVG-іконок, локальних шрифтів, JSON-файлів та ES6 модулів (<script type="module">), які за замовчуванням блокуються в режимі file:///.

7. Що таке Prettier і чому його рекомендують використовувати? Які основні правила форматування він застосовує?

Prettier — це безапеляційний (opinionated) інструмент автоформатування вихідного коду. Він парсить код у абстрактне синтаксичне дерево (AST) та друкує його заново, повністю стандартизуючи зовнішній вигляд.
Чому його рекомендують: він знімає необхідність витрачати час на ручне вирівнювання рядків, нівелює суперечки в команді щодо стилю кодування та очищає історію комітів Git від випадкових пробільних змін.
Основні правила: застосування чітких відступів (2 або 4 пробіли замість Tab), дотримання ліміту

довжини рядка (Print Width) з автоматичним перенесенням атрибутів, правильне форматування дужок, ком та пробілів навколо операторів і селекторів.

8. Наведіть приклад використання CSS Flexbox для створення горизонтального меню. Які ключові властивості ви використовували?

У проєкті ChargeUA горизонтальне меню створено на базі Flexbox:

```
.nav__list {
  display: flex;
  align-items: center;
  gap: var(--space-1) var(--space-3);
  list-style: none;
}
.nav__link {
  display: inline-flex;
  align-items: center;
  gap: var(--space-2);
  min-height: 44px; /* Мінімальний розмір клікабельної зони WCAG */
}
```

Ключові властивості:

- display: flex; — вмикає гнучкий одновимірний контекст розташування елементів.
- align-items: center; — центрує елементи меню по вертикальній осі.
- gap: — задає відступ між посиланнями без використання margin-right.
- flex-wrap: wrap; — дозволяє меню коректно переноситися на вузьких екранах.

9. Наведіть приклад використання CSS Grid для створення сітки з 3 колонок. Яка різниця між Flexbox і Grid?

У проєкті ChargeUA для карток каталогу застосовано адаптивний CSS Grid:

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 24px;
}
```

Різниця між Flexbox та Grid:

1) Вимірність: Flexbox — одновимірна система (1D), керує елементами або в рядок, або в стовпчик. Grid — двовимірна система (2D), керує рядками і стовпцями одночасно.
2) Логіка компонування: Flexbox компонує макет від контенту (content-out), а Grid формує просторову структуру сітки, в яку потім поміщається контент (layout-in).
3) Застосування: Flexbox підходить для шапок, панелей дій та мікровирівнювання; Grid — для галерей, каталогів карток та складних макросіток сайту.

10. Як правильно організувати архітектуру CSS-змінних? Наведіть приклад структури :root.

Архітектура CSS-змінних організовується за принципом рівнів абстракції (токени бренду → функціональні семантичні змінні → змінні компонентів):

```
:root {
  /* Брендові токени (Colors) */
  --color-primary: #005EC9;
  --color-secondary: #0F172A;
```

```
  --color-accent:                                                    #00B4D8;
  --color-bg:                                                        #FFFFFF;
  --color-surface:                                                   #F8FAFC;
  /*                                  Статуси                                */
  --color-status-free:                                               #10B981;
  --color-status-busy:                                               #EF4444;
  --color-status-reserved:                                           #F59E0B;
  /*                                 Типографіка                             */
  --font-base:          'Inter',          system-ui,                 sans-serif;
  --fs-base:                                                         1rem;
  --fs-xl:                                                           1.75rem;
  /*          Відступи,          радіуси,          тіні                    */
  --space-2:    0.5rem;    --space-4:    1rem;    --space-6:    2rem;
  --radius-sm:  6px;       --radius-md:  12px;    --radius-pill:  999px;
  --shadow-sm:  0  1px  3px  rgb(15  23  42  /  0.08);
  --container-max:                                                   1180px;
}
```

11. Що таке рефакторинг (Refactoring Guard)? Опишіть основні етапи цієї методології.

Рефакторинг — це контрольована оптимізація внутрішньої структури кодової бази без зміни зовнішньої функціональності та візуального вигляду інтерфейсу. Методологія Refactoring Guard гарантує збереження стабільності під час трансформацій.
Етапи методології:
1. Фіксація еталону (Baseline): Створення резервної копії несемантичного стану коду та зняття візуальних референсів.
2. Аудит якості: Виявлення порушень стандартів W3C, div-мішанини та дублювання CSS-властивостей.
3. Ітераційне впровадження змін: Покроковий перехід на семантичні вузли HTML5 та винесення стильових констант у токени.
4. Валідація та верифікація: Перевірка через W3C Markup Validator, тестування доступності Lighthouse та контроль відсутності візуальних регресій.

12. Як правильно скласти промпт для ШІ, щоб він виконав якісний рефакторинг CSS-коду? Наведіть приклад ефективного промпту.

Правильний промпт повинен містити:
1) Рольовий контекст (Senior CSS Architect).
2) Точне технічне завдання (винести дублікати в :root, зберегти адаптивність).
3) Негативні обмеження (жодних сторонніх фреймворків Bootstrap/Tailwind, не змінювати зовнішній вигляд).
4) Очікуваний результат (список змін + цілісний код).
Приклад ефективного промпту:
«Дій як Senior CSS Engineer. Проаналізуй наданий CSS-код сторінки index.html. Виконай рефакторинг згідно з принципом DRY. 1. Вияви всі дублювання кольорів та відступів і згрупуй їх у :root зі змістовними назвами. 2. Для секції карток реалізуй адаптивний CSS Grid з repeat(auto-fit, minmax(280px, 1fr)). 3. Не використовуй Tailwind чи Bootstrap. 4. Збережи поточний візуальний дизайн і HTML-структуру. Надай лаконічний звіт про внесені оптимізації та фінальний файл style.css».

13. В чому полягає принцип DRY (Don’t Repeat Yourself) в контексті CSS? Як ШІ допомагає його дотримуватися?

Принцип DRY у CSS означає усунення будь-якого дублювання декларацій властивостей і значень. Замість багаторазового прописування однакових значень (кольорів #005EC9, падінгів 16px, тіней) вони виносяться в єдине джерело правди — змінні :root.
Крім того, DRY забезпечується через класи компонентів: базовий клас .btn містить спільну геометрію та інтерактивні стани, а модифікатори .btn--accent, .btn--outline додають лише кольорову специфіку.
Як ШІ допомагає: асистент миттєво знаходить повторювані блоки стилів у великих файлах, пропонує оптимальну ієрархію токенів та автоматично переписує селектори, видаляючи надлишкові правила.

14. Які переваги надає використання ШІ-асистентів (Copilot/Codeium) під час верстки порівняно з ручним написанням коду? Наведіть конкретні приклади з лабораторної.

Переваги використання ШІ-асистентів:
• Прискорення розробки: Миттєва кодогенерація складних розміток, списків, форм пошуку та карток послуг.
• Підвищення якості та валідності: ШІ одразу підказує семантичні альтернативи для div, автоматично генерує атрибути доступності (aria-label, role, alt, skip-link).
• Автоматизований рефакторинг: Можливість комплексно переписати 300 рядків div-мішанини на чистий семантичний HTML5 за лічені секунди.
Конкретний приклад з лабораторної ChargeUA: ШІ-асистент Claude успішно побудував систему компонентних статусів портів (.status, .status--busy, .status--reserved) на базі сучасної функції color-mix(in srgb, var(--status) 16%, var(--color-bg)), що дозволило керувати кольором мітки, текстом та фоном за допомогою зміни лише одного токена --status, скоротивши обсяг CSS-коду на 60%.

10. Висновки

Під час виконання лабораторної роботи №5 успішно вирішено поставлені завдання з проектування сучасної семантичної верстки та рефакторингу базового каркасу вебплатформи «ChargeUA»:

1. Опановано та налаштовано повний стек інструментів робочого середовища VS Code, включаючи розширення Live Server, Prettier, Auto Rename Tag, Material Icon Theme та агентні ШІ-інструменти (Claude, Gemini, Antigravity).

2. На практиці проведено навчальний експеримент з аналізу та виявлення проблем несемантичної верстки («div soup»), зафіксовано деградацію дерева доступності (Accessibility Tree) та неможливість повноцінної орієнтації екранних дикторів на несемантичних сторінках.

3. Виконано повноцінний ШІ-рефакторинг файлу index.html, впроваджено 100% семантичну структуру стандарту HTML5 (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>, <address>, <dl>) з суворою ієрархією заголовків h1–h3 та дотриманням WCAG 2.2 AA (клавіатурний skip-link, приховані орієнтири для скрінрідерів, контрастні пари кольорів).

4. Спроектовано та реалізовано професійну дворівневу CSS-архітектуру без використання сторонніх фреймворків: усі стильові константи бренду ChargeUA винесено в глобальні токени :root, побудовано адаптивні макети на Flexbox та адаптивній сітці CSS Grid з repeat(auto-fit, minmax(280px, 1fr)), інтегровано Google Font 'Inter' та іконки Font Awesome 6.5.

5. За допомогою інструментів Chrome DevTools та Lighthouse Snapshot Audit підтверджено бездоганну якість верстки: досягнуто найвищих оцінок Accessibility: 100 / 100 та Best Practices: 100 / 100.

6. Складено структурований Промпт-паспорт і надано ґрунтовні відповіді на всі 14 контрольних запитань.